

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 12 – SEARCHING



NAMA : Wahid Satrio Aji

NIM : 109082500005

KELAS S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Pencarian data (searching) merupakan operasi fundamental dalam pemrograman untuk menemukan suatu data spesifik di dalam sekumpulan himpunan data, seperti mencari mahasiswa dengan nama atau NIM tertentu. Berbeda dengan pencarian nilai ekstrem (minimum/maksimum) yang nilainya selalu ada, pada operasi pencarian spesifik terdapat kemungkinan bahwa data yang dicari tidak ditemukan di dalam himpunan tersebut. Untuk menangani permasalahan ini, terdapat dua algoritma utama yang umum digunakan, yakni Sequential Search dan Binary Search.

Algoritma Sequential Search bekerja dengan cara melakukan perbandingan elemen data secara berurutan dan satu persatu, mulai dari data pertama hingga data terakhir. Ciri khas utama dari algoritma ini adalah proses iterasi pencarian akan langsung dihentikan (terminasi) seketika ketika data yang dicari telah berhasil ditemukan, meskipun masih terdapat sisa data di dalam himpunan yang belum diperiksa nilainya. Algoritma ini sangat fleksibel karena dapat diterapkan pada kumpulan data yang tidak terurut.

Sementara itu, Binary Search adalah algoritma pencarian yang jauh lebih efisien namun memiliki prasyarat mutlak, yaitu himpunan data harus sudah dalam keadaan terurut, misalnya terurut membesar (ascending). Algoritma ini beroperasi secara logaritmik dengan menentukan batas kiri dan batas kanan, lalu mengambil elemen paling tengah (median) di dalam rentang tersebut sebagai titik perbandingan. Apabila nilai yang dicari lebih besar dari nilai tengah, maka rentang pencarian akan digeser dengan mengubah batas kiri menjadi posisi setelah elemen tengah. Sebaliknya, jika data lebih kecil, batas kanan yang akan digeser.

Kedua algoritma pencarian ini juga dapat diekspansi untuk memproses sekumpulan data terstruktur yang dibungkus menggunakan tipe data struct. Implementasinya dilakukan dengan menambahkan spesifikasi field atau atribut kategori yang ingin dicari. Hal krusial yang perlu diperhatikan saat mengimplementasikan Binary Search pada struct adalah keterurutan elemen di dalam array harus bersesuaian persis dengan field yang menjadi kunci pencarian tersebut.

B. Guided

1. Sequential Search

```
package main

import "fmt"

type arrData [5]string //tipe data alias

func seqSearch(arr arrData, binatangCari string) int {
    var found int = -1
    for i := 0; i < len(arr); i++ {
        if arr[i] == binatangCari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var arrBinatang arrData //array tipe string

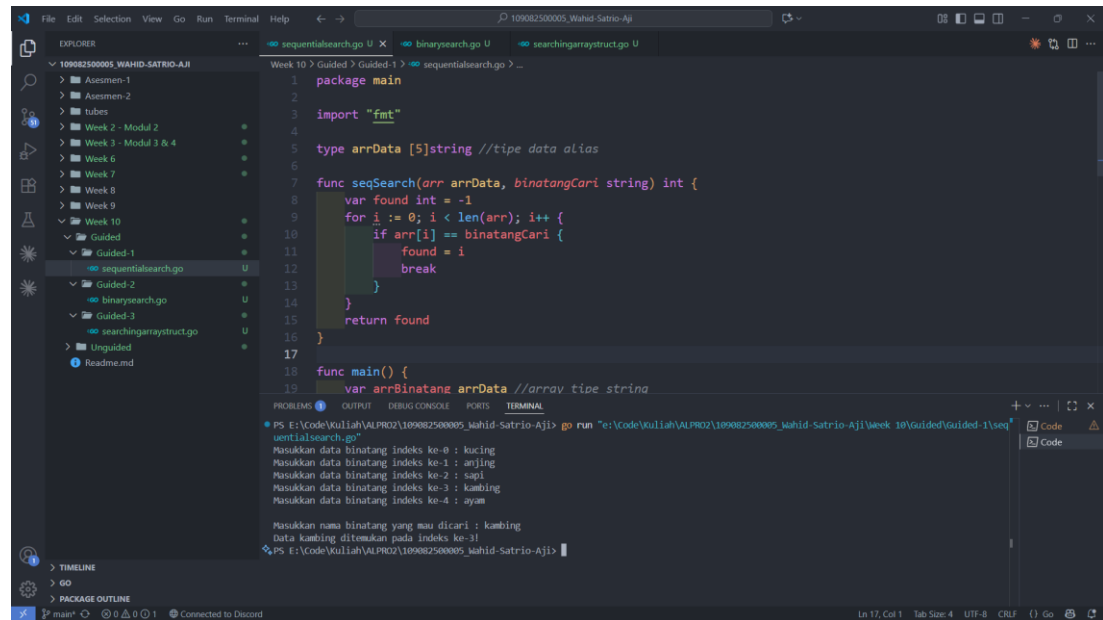
    for i := 0; i < len(arrBinatang); i++ {
        fmt.Printf("Masukkan data binatang indeks ke-%d : ", i)
        fmt.Scan(&arrBinatang[i])
    }
    fmt.Println()

    var binatangCari string
    fmt.Print("Masukkan nama binatang yang mau dicari : ")
    fmt.Scan(&binatangCari)

    var idxCari int
    idxCari = seqSearch(arrBinatang, binatangCari)

    if idxCari > -1 {
        fmt.Printf("Data %s ditemukan pada indeks ke-%d!", binatangCari, idxCari)
    } else if idxCari == -1 {
        fmt.Printf("Data %s tidak ditemukan!", binatangCari)
    }
}
```

Output :



```
1 package main
2
3 import "fmt"
4
5 type arrData [5]string // tipe data alias
6
7 func seqSearch(arr arrData, binatangCari string) int {
8     var found int = -1
9     for i := 0; i < len(arr); i++ {
10         if arr[i] == binatangCari {
11             found = i
12             break
13         }
14     }
15     return found
16 }
17
18 func main() {
19     var arrBinatang arrData // array tipe string
20
21     // ... (input handling code) ...
22
23     // ... (calling seqSearch and printing result) ...
24 }
```

Terminal Output:

```
PS E:\Code\Yuliah\ALPRO2\109082500005_Mahid-Satrio-Aji> go run "E:\Code\Yuliah\ALPRO2\109082500005_Mahid-Satrio-Aji\Week 10\Guided\Guided-1\SequentialSearch.go"
Masukkan data binatang indeks ke-0 : kucing
Masukkan data binatang indeks ke-1 : anjing
Masukkan data binatang indeks ke-2 : sapi
Masukkan data binatang indeks ke-3 : kambing
Masukkan data binatang indeks ke-4 : ayam

Masukkan nama binatang yang mau dicari : kambing
data kambing ditemukan pada indeks ke-3!
PS E:\Code\Yuliah\ALPRO2\109082500005_Mahid-Satrio-Aji>
```

Penjelasan :

Program ini bertujuan untuk mencari posisi indeks dari sebuah data bertipe string (dalam kasus ini adalah nama binatang) di dalam sebuah array statis menggunakan metode Sequential Search. Program memisahkan logika pencarian ke dalam subprogram berwujud fungsi `seqSearch` yang menerima parameter berupa array data dan string target yang ingin dicari. Di dalam fungsi tersebut, program menginisialisasi variabel pelacak status (`found`) dengan nilai `-1` yang diasumsikan sebagai flag jika data tidak ditemukan. Selanjutnya, iterasi perulangan dijalankan secara linear dari indeks ke-0 hingga indeks terakhir array untuk menguji kesamaan (equality) antara elemen array saat ini dengan string target. Apabila kondisi kecocokan terpenuhi, program langsung memperbarui nilai `found` dengan indeks lokasi elemen tersebut, dan seketika itu juga mengeksekusi instruksi `break` untuk menghentikan perulangan lebih awal tanpa perlu memeriksa sisa data. Pada blok fungsi utama (`main`), program melakukan iterasi untuk mengumpulkan input 5 nama binatang dari pengguna, meminta nama binatang yang ingin dicari, lalu memanggil fungsi `seqSearch` untuk mengevaluasi dan mencetak hasil temuan lokasi indeksnya ke layar.

2. Binary Search

```
package main

import "fmt"

// fungsi binary search
func binarySearch(data []int, x int) int {

    // batas kiri array
    kiri := 0

    // batas kanan array
    kanan := len(data) - 1

    // perulangan selama kiri <= kanan
    for kiri <= kanan {

        // mencari index tengah
        tengah := (kiri + kanan) / 2

        // jika data ditemukan
        if data[tengah] == x {
            return tengah
        }

        // jika x lebih besar, cari ke kanan
        } else if data[tengah] < x {
            kiri = tengah + 1

        // jika x lebih kecil, cari ke kiri
        } else {
            kanan = tengah - 1
        }
    }

    // jika data tidak ditemukan
    return -1
}

func main() {

    var n int
    var cari int

    // input jumlah data
    fmt.Print("Jumlah data: ")
    fmt.Scan(&n)

    // membuat array sesuai jumlah data
    data := make([]int, n)
```

```

// input isi array
fmt.Println("Masukkan data terurut:")
for i := 0; i < n; i++ {
    fmt.Scan(&data[i])
}

// input angka yang dicari
fmt.Print("Angka yang dicari: ")
fmt.Scan(&cari)

// memanggil fungsi binary search
hasil := binarySearch(data, cari)

// output hasil pencarian
if hasil != -1 {
    fmt.Println("Data ditemukan di index:", hasil)
} else {
    fmt.Println("Data tidak ditemukan")
}
}

```

Output :

The screenshot shows a VS Code editor with a Go file named `binarysearch.go` in the `Week 10 > Guided > Guided-2` directory. The code implements a binary search function. The terminal output shows the program's execution:

```

PS E:\Code\Wuliah\VALPRO2\109082500005_Mahid-Satrio-Aji> go run "e:\Code\Wuliah\VALPRO2\109082500005_Mahid-Satrio-Aji\Week 10\Guided\Guided-2\bin\
anysearch.go"
Jumlah data: 7
Masukkan data terurut:
1 3 5 7 9 11 13
Angka yang dicari: 9
Data ditemukan di index: 4
PS E:\Code\Wuliah\VALPRO2\109082500005_Mahid-Satrio-Aji>

```

Penjelasan :

Program ini bertujuan untuk mengidentifikasi keberadaan dan lokasi indeks suatu bilangan bulat di dalam sebuah array dinamis (slice) menggunakan pendekatan reduksi ruang pencarian melalui algoritma Binary Search. Berbeda dengan pencarian linear, program ini mendefinisikan sebuah fungsi bernama `binarySearch` yang mengoperasikan tiga titik penunjuk indeks: kiri yang dimulai dari angka nol, kanan yang berada di batas akhir slice, dan variabel tengah yang selalu dikalkulasi secara dinamis membagi dua sisa rentang pencarian. Di dalam siklus perulangannya, program membandingkan nilai target dengan elemen pada indeks tengah. Jika nilai elemen di posisi tengah tersebut kurang dari target pencarian, maka rentang kiri akan dieliminasi dan batas kiri digeser menjadi $\text{tengah} + 1$. Sebaliknya, jika target pencarian lebih kecil, rentang kanan yang akan dibuang dengan memperbarui batas kanan menjadi $\text{tengah} - 1$. Proses isolasi logaritmik ini terus berlangsung hingga elemen yang tepat berhasil ditemukan dan program mengembalikan koordinat tengah tersebut, atau hingga rentang batas saling bersilangan yang akan mengembalikan flag `-1`. Pada fungsi utamanya, program berinteraksi dengan pengguna untuk mengalokasikan ukuran slice, menerima himpunan data yang dikondisikan harus terurut, serta mengeksekusi pencarian secara presisi.

3. Searching pada array struct

```
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim   int
    IPK   float64
}

type arrMahasiswa [5]mahasiswa

func binSearchStruct(arrData arrMahasiswa, nimDicari int)
int {
    var found int = -1
    var med int
    var kr int = 0
    var kn int = len(arrData) - 1
    for kr <= kn && found == -1 {
        med = (kr + kn) / 2
        if nimDicari < arrData[med].nim {
            kn = med - 1
        } else if nimDicari > arrData[med].nim {
            kr = med + 1
        } else {
            found = med
        }
    }
    return found
}

func seqSearchStruct(arrData arrMahasiswa, nimDicari int)
int {
    var found int = -1
    for i := 0; i < len(arrData); i++ {
        if arrData[i].nim == nimDicari {
            found = i
            break
        }
    }
    return found
}

func main() {
    var mahasiswa06 arrMahasiswa
    var nimDicari int

    fmt.Println("--- MASUKKAN DATA NIM MAHASISWA SECARA
    TERURUT ---")
}
```



```

        for i := 0; i < len(mahasiswa06); i++ {
            fmt.Printf("=== Masukkan data mahasiswa 06 pada
indeks ke-%d ===\n", i)
            fmt.Print("Masukkan nama : ")
            fmt.Scan(&mahasiswa06[i].nama)
            fmt.Print("Masukkan NIM : ")
            fmt.Scan(&mahasiswa06[i].nim)
            fmt.Print("Masukkan IPK : ")
            fmt.Scan(&mahasiswa06[i].IPK)
            fmt.Println("-----")
        }
        fmt.Println()

        fmt.Print("Masukkan NIM mahasiswa yang ingin dicari :
")
        fmt.Scan(&nimDicari)
        fmt.Println()

        var idxHasilCariSeqSearch int
        idxHasilCariSeqSearch = seqSearchStruct(mahasiswa06,
nimDicari)

        fmt.Println("== HASIL SEQUENTIAL SEARCH ==")
        if idxHasilCariSeqSearch > -1 {
            fmt.Printf("Data NIM mahasiswa %d ditemukan pada
array di indeks ke-%d!\n", nimDicari, idxHasilCariSeqSearch)
            fmt.Println("Berikut Detail Datanya : ")
            fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariSeqSearch].nama)
            fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariSeqSearch].nim)
            fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariSeqSearch].IPK)
        } else if idxHasilCariSeqSearch == -1 {
            fmt.Printf("Data NIM mahasiswa %d tidak ditemukan
pada array!", nimDicari)
        }
        fmt.Println()

        var idxHasilCariBinSearch int
        idxHasilCariBinSearch = binSearchStruct(mahasiswa06,
nimDicari)

        fmt.Println("== HASIL BINARY SEARCH ==")
        if idxHasilCariBinSearch > -1 {
            fmt.Printf("Data NIM mahasiswa %d ditemukan pada
array di indeks ke-%d!\n", nimDicari, idxHasilCariBinSearch)
            fmt.Println("Berikut Detail Datanya : ")
            fmt.Printf("Nama : %s\n",
mahasiswa06[idxHasilCariBinSearch].nama)

```

```

        fmt.Printf("NIM : %d\n",
mahasiswa06[idxHasilCariBinSearch].nim)
        fmt.Printf("IPK : %.2f\n",
mahasiswa06[idxHasilCariBinSearch].IPK)
    } else if idxHasilCariBinSearch == -1 {
        fmt.Printf("Data NIM mahasiswa %d tidak ditemukan
pada array!", nimDicari)
    }
}

```

Output :

The screenshot shows a Go IDE with the following code in `searchingarraystruct.go`:

```

package main

import "fmt"

type mahasiswa struct {
    nama string
    nim   int
    ipk   float64
}

type arrMahasiswa [5]mahasiswa

func binSearchStruct(arrData arrMahasiswa, nimDicari int) int {
    var found int = -1
    var med int

```

The terminal output shows the execution of the program:

```

PS E:\Code\Kuliah\ALPRO2\109082500005_Mahid-Satrio-Aji> go run "e:\Code\Kuliah\ALPRO2\109082500005_Mahid-Satrio-Aji\Week 10\Guided-3\searchingarraystruct.go"
=== Masukkan data mahasiswa 06 pada indeks ke-3 ===
Masukkan nama : Dedi
Masukkan NIM : 1004
Masukkan IPK : 3.6
=====
=== Masukkan data mahasiswa 06 pada indeks ke-4 ===
Masukkan nama : Eka
Masukkan NIM : 1005
Masukkan IPK : 3.9
=====
Masukkan NIM mahasiswa yang ingin dicari : 1003

== HASIL SEQUENTIAL SEARCH ==
Data NIM mahasiswa 1003 ditemukan pada array di indeks ke-2!
Berikut Detail Datanya :
Nama : Cici
NIM : 1003
IPK : 3.80

== HASIL BINARY SEARCH ==
Data NIM mahasiswa 1003 ditemukan pada array di indeks ke-2!
Berikut Detail Datanya :
Nama : Cici
NIM : 1003
IPK : 3.80

```

Penjelasan :

Program ini bertujuan untuk menerapkan dan mengkomparasi dua algoritma pencarian data. Sequential Search dan Binary Search secara bersamaan di dalam ekosistem tipe data bentukan (struct). Di tahap awal, program membungkus tiga atribut record berbeda, yaitu nama, nim, dan IPK ke dalam sebuah entitas struct bernama mahasiswa, kemudian membentuk tipe data alias `arrMahasiswa` untuk menyediakan array statis berkapasitas lima mahasiswa. Logika program dienkapsulasi ke dalam dua fungsi terpisah: `seqSearchStruct` yang mencari target NIM dengan menelusuri indeks secara berurutan, dan `binSearchStruct` yang melokalisasi target NIM dengan cara membelah dua batas indeks pencariannya. Karakteristik krusial yang ditunjukkan pada kode pencarian ini adalah penggunaan dot notation (`arrData[i].nim` dan `arrData[med].nim`), yang menginstruksikan program untuk secara eksplisit

membidik dan mengekstrak field spesifik (NIM) dari dalam bongkahan objek struct sebagai acuan komparasi. Pada fungsi utama (main), program melakukan proses penginputan data terstruktur dari pengguna yang diasumsikan langsung terurut membesar secara terstruktur, mengeksekusi kedua fungsi pencarian secara paralel terhadap kunci NIM yang sama, dan menampilkan validasi pencarian beserta penjabaran detail komponen atribut struct secara menyeluruh apabila NIM mahasiswa yang dicari berhasil ditemukan pada ruang array tersebut.

C. Unguided

1. Soal Unguided 1

- 1) Pada pemilihan ketua RT yang baru saja berlangsung, terdapat 20 calon ketua yang bertanding memperebutkan suara warga. Perhitungan suara dapat segera dilakukan karena warga cukup mengisi formulir dengan nomor dari calon ketua RT yang dipilihnya. Seperti biasa, selalu ada pengisian yang tidak tepat atau dengan nomor pilihan di luar yang tersedia, sehingga data juga harus divalidasi. Tugas Anda untuk membuat program mencari siapa yang memenangkan pemilihan ketua RT.

Buatlah program **pilkart** yang akan membaca, memvalidasi, dan menghitung suara yang diberikan dalam pemilihan ketua RT tersebut.

Jawaban :

```
package main

import (
    "fmt"
)

func main() {
    var suara [21]int
    var masuk, sah, vote int

    for {
        fmt.Scan(&vote)
        if vote == 0 {
            break
        }

        masuk++

        if vote >= 1 && vote <= 20 {
            sah++
            suara[vote]++
        }
    }
}
```

```

    }
}

fmt.Printf("Suara masuk: %d\n", masuk)
fmt.Printf("Suara sah: %d\n", sah)

for i := 1; i <= 20; i++ {
    if suara[i] > 0 {
        fmt.Printf("%d: %d\n", i, suara[i])
    }
}
}

```

Output :

The screenshot shows a Go IDE with the following code in the editor:

```

package main

import (
    "fmt"
)

func main() {
    var suara [21]int
    var masuk, sah, vote int

    for {
        fmt.Scan(&vote)
        if vote == 0 {
            break
        }
        masuk++

        if vote >= 1 && vote <= 20 {

```

The terminal output shows the execution of the program:

```

PS E:\Code\Kulia\ALPRO2\109082500005_Wahid-Satrio-Aji> go run "e:\Code\Kulia\ALPRO2\109082500005_Wahid-Satrio-Aji\Week 10\Unguided\Unguided 1"
7 19 3 2 78 3 1 -3 18 19 0
Suara masuk: 10
Suara sah: 8
1: 1
2: 1
3: 2
7: 1
18: 1
19: 2

```

Penjelasan :

Program ini bertujuan untuk membaca, memvalidasi, serta melakukan tabulasi terhadap data perolehan suara dalam pemilihan ketua RT. Program memanfaatkan sebuah array statis bernama suara berkapasitas 21 elemen bertipe integer yang berfungsi sebagai tabel frekuensi, di mana indeks 1 hingga 20 merepresentasikan nomor urut masing-masing kandidat secara spesifik. Proses penginputan data dilakukan melalui mekanisme perulangan tak terbatas yang akan terus membaca aliran data masukan hingga mendeteksi angka nol sebagai kondisi terminasi (break). Di dalam perulangan tersebut, setiap elemen suara yang masuk akan dihitung ke dalam variabel akumulator masuk, lalu dilakukan tahapan seleksi kondisi untuk memastikan nomor kandidat berada

pada rentang valid antara 1 hingga 20. Apabila suara dinyatakan valid, variabel sah akan bertambah dan nilai array pada indeks yang bersesuaian akan diinkrementasi secara otomatis sebagai bentuk pencatatan suara sah. Setelah seluruh data selesai diproses oleh sistem, program pada blok fungsi utama (main) akan mencetak total suara masuk dan suara sah, kemudian melakukan iterasi sekuensial dari indeks 1 sampai 20 untuk menampilkan nomor urut kandidat beserta akumulasi perolehan suaranya yang bernilai lebih dari nol ke layar komputer.

2. Soal Unguided 2

- 2) Berdasarkan program sebelumnya, buat program **pilkart** yang mencari siapa pemenang pemilihan ketua RT. Sekaligus juga ditentukan bahwa wakil ketua RT adalah calon yang mendapatkan suara terbanyak kedua. Jika beberapa calon mendapatkan suara terbanyak yang sama, ketua terpilih adalah dengan nomor peserta yang paling kecil dan wakilnya dengan nomor peserta terkecil berikutnya.

Jawaban :

```
package main

import (
    "fmt"
)

func main() {
    var suara [21]int
    var masuk, sah, vote int

    for {
        fmt.Scan(&vote)
        if vote == 0 {
            break
        }
        masuk++
        if vote >= 1 && vote <= 20 {
            sah++
            suara[vote]++
        }
    }

    fmt.Printf("Suara masuk: %d\n", masuk)
    fmt.Printf("Suara sah: %d\n", sah)
```

```

var ketua, wakil int
var maxKetua, maxWakil int

for i := 1; i <= 20; i++ {
    if suara[i] > maxKetua {
        wakil = ketua
        maxWakil = maxKetua

        ketua = i
        maxKetua = suara[i]
    } else if suara[i] > maxWakil {
        wakil = i
        maxWakil = suara[i]
    } else if suara[i] == maxKetua && ketua != 0 &&
    wakil == 0 {
        wakil = i
        maxWakil = suara[i]
    } else if suara[i] == maxKetua && suara[i] >
    maxWakil {
        wakil = i
        maxWakil = suara[i]
    }
}

fmt.Printf("Ketua RT: %d\n", ketua)
fmt.Printf("Wakil ketua: %d\n", wakil)
}

```

Output :

The screenshot shows a VS Code editor with a Go project. The file explorer on the left shows a directory structure with folders like 'Asesmen-1', 'Asesmen-2', 'tubes', and 'Week 2 - Modul 2'. The main editor window shows a Go file named 'ketua&wakil.go' with the following code:

```

1 package main
2
3 import (
4     "fmt"
5 )
6
7 func main() {
8     var suara [21]int
9     var masuk, sah, vote int
10
11     for {
12         fmt.Scan(&vote)
13         if vote == 0 {
14             break
15         }
16         masuk++
17         if vote >= 1 && vote <= 20 {
18             sah++
19             suara[vote]++
20         }
21     }
22 }

```

The terminal at the bottom shows the output of the program:

```

PS E:\Code\Kuliah\ALPRO2\109082500005_Wahid-Satrio-Aji> go run ".\ketua&wakil.go"
7 19 3 2 18 3 1 -3 18 19 0
Suara masuk: 19
Suara sah: 8
Ketua RT: 3
Wakil ketua: 19
PS E:\Code\Kuliah\ALPRO2\109082500005_Wahid-Satrio-Aji>

```

Penjelasan :

Program ini bertujuan untuk memperluas sistem tabulasi suara sebelumnya dengan menambahkan logika pencarian nilai ekstrem sekuensial (Sequential Search) guna menentukan kandidat yang terpilih sebagai Ketua RT dan Wakil Ketua RT. Program menginisialisasi empat variabel pelacak baru, yaitu ketua dan wakil untuk menyimpan nomor urut kandidat pemenang, serta maxKetua dan maxWakil untuk menampung jumlah akumulasi suara tertinggi pertama dan kedua. Setelah seluruh data masukan berhasil dikumpulkan ke dalam array tabel frekuensi, program melakukan pencarian nilai ekstrem dengan mengiterasi array dari indeks 1 hingga 20. Pada setiap tahapan iterasi, program mengevaluasi data menggunakan seleksi kondisi berlapis: apabila ditemukan kandidat dengan perolehan suara yang melampaui maxKetua, maka data ketua saat ini akan dialihkan menjadi wakil, lalu posisi ketua dan nilai maxKetua diperbarui dengan data kandidat baru tersebut. Namun, jika suara kandidat tidak melampaui ketua melainkan hanya melampaui maxWakil, atau memenuhi kondisi imbang (tie-breaker) dengan aturan nomor urut terkecil yang didahulukan melalui sifat iterasi ascending, maka posisi wakil dan nilai maxWakil yang akan diperbarui secara bersamaan. Pada bagian akhir blok fungsi utama (main), program mengeksekusi perintah keluaran untuk menampilkan nomor urut kandidat yang berhasil menempati posisi jabatan Ketua RT dan Wakil Ketua RT berdasarkan hasil kalkulasi akhir.

3. Soal Unguided 3

- 3) Diberikan n data integer positif dalam keadaan terurut membesar dan sebuah integer lain k, apakah bilangan k tersebut ada dalam daftar bilangan yang diberikan? Jika ya, berikan indeksnya, jika tidak sebutkan "TIDAK ADA".

Jawaban :

```
package main

import (
    "fmt"
)

func main() {
    var suara [21]int
    var masuk, sah, vote int
```

```

    for {
        fmt.Scan(&vote)
        if vote == 0 {
            break
        }
        masuk++
        if vote >= 1 && vote <= 20 {
            sah++
            suara[vote]++
        }
    }

    fmt.Printf("Suara masuk: %d\n", masuk)
    fmt.Printf("Suara sah: %d\n", sah)

    var ketua, wakil int
    var maxKetua, maxWakil int

    for i := 1; i <= 20; i++ {
        if suara[i] > maxKetua {
            wakil = ketua
            maxWakil = maxKetua

            ketua = i
            maxKetua = suara[i]
        } else if suara[i] > maxWakil {
            wakil = i
            maxWakil = suara[i]
        } else if suara[i] == maxKetua && ketua != 0 &&
    wakil == 0 {
            wakil = i
            maxWakil = suara[i]
        } else if suara[i] == maxKetua && suara[i] >
    maxWakil {
            wakil = i
            maxWakil = suara[i]
        }
    }

    fmt.Printf("Ketua RT: %d\n", ketua)
    fmt.Printf("Wakil ketua: %d\n", wakil)
}

```

Output :


```
1  import "fmt"
2
3  const NMAX = 1000000
4  var data [NMAX]int
5
6  func isiArray(n int) {
7      for i := 0; i < n; i++ {
8          fmt.Scan(&data[i])
9      }
10 }
11
12 func posisi(n, k int) int {
13     var kr int = 0
14     var kn int = n - 1
15     var found int = -1
16     var med int
17
18     for kr <= kn && found == -1 {
19         med = (kr + kn) / 2
20     }
21 }
```

PS E:\Code\Uuliah\ALPRO2\109082500005_wahid-Satrio-Aji\Week 10\Unguided\Unguided-3> go run "e:\Code\Uuliah\ALPRO2\109082500005_wahid-Satrio-Aji\Week 10\Unguided\Unguided-3\implementbiner.go"

12 534

1 3 8 16 32 123 323 323 534 543 823 999

8

PS E:\Code\Uuliah\ALPRO2\109082500005_wahid-Satrio-Aji\Week 10\Unguided\Unguided-3> go run "e:\Code\Uuliah\ALPRO2\109082500005_wahid-Satrio-Aji\Week 10\Unguided\Unguided-3\implementbiner.go"

12 535

1 3 8 16 32 123 323 323 534 543 823 999

TIDAK ADA

PS E:\Code\Uuliah\ALPRO2\109082500005_wahid-Satrio-Aji\Week 10\Unguided\Unguided-3>

Penjelasan :

Program ini bertujuan untuk mencari posisi indeks dari suatu bilangan bulat di dalam kumpulan data terurut berskala besar menggunakan algoritma pencarian biner (Binary Search). Program menerapkan pendekatan modular yang ketat dengan mengalokasikan sebuah array global bernama data berkapasitas satu juta elemen, serta membagi instruksi ke dalam prosedur isiArray dan fungsi posisi. Prosedur isiArray berfungsi melakukan iterasi sekuensial untuk memasukkan sejumlah n data bilangan dari piranti masukan ke dalam memori array global. Sementara itu, fungsi posisi menerima parameter jumlah data n dan nilai target k, lalu menginisialisasi batas kiri (kr) di angka nol, batas kanan (kn) di indeks n-1, serta penanda status penemuan (found) bernilai negatif satu. Di dalam fungsi ini, perulangan berjalan secara logaritmik dengan membagi dua ruang pencarian secara terus-menerus untuk menentukan koordinat titik tengah (median). Batas kanan atau batas kiri akan digeser secara dinamis menjauhi median berdasarkan hasil perbandingan nilai k dengan nilai elemen di posisi median hingga data berhasil ditemukan atau ruang pencarian habis bertindihan. Pada blok fungsi utama (main), setelah membaca nilai n dan k serta mengeksekusi kedua subprogram tersebut, program akan mengevaluasi nilai kembalian (return value) fungsi untuk mencetak teks "TIDAK ADA" jika data tidak ditemukan, atau mencetak angka indeks lokasi persisnya jika data berhasil diidentifikasi di dalam array.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa bahasa Go sangat mampu mengimplementasikan logika pencarian (searching) kompleks untuk melokalisasi data yang spesifik. Algoritma Sequential Search terbukti menjadi solusi yang sangat fleksibel dan aman (meskipun memerlukan iterasi yang lebih banyak) karena dapat beroperasi pada himpunan data mentah yang belum terurut. Algoritma ini memastikan tidak ada data yang terlewat karena pengecekan dilakukan satu persatu.

Di sisi lain, Binary Search mendemonstrasikan efisiensi pencarian tingkat tinggi dengan memangkas setengah ruang komputasi pada setiap iterasinya. Namun, efisiensi ini diiringi dengan prasyarat arsitektur program yang ketat, di mana pengembang (developer) secara absolut diwajibkan untuk menjamin bahwa himpunan data telah melalui proses pengurutan (sorting) terlebih dahulu sebelum pencarian biner dapat dieksekusi

Praktikum ini juga memvalidasi fleksibilitas kedua algoritma tersebut yang tidak hanya terbatas pada pencarian array tipe data primitif, melainkan dapat diintegrasikan pada tipe data rekaman (struct). Dengan memanfaatkan notasi titik (.), algoritma pencarian mampu menginspeksi entitas data skala riil yang kompleks (seperti NIM atau metrik performa mahasiswa) dengan menjadikan field atribut spesifik di dalam struct sebagai kunci indeks pencarian sentral (search key). Pemanfaatan modularitas melalui subprogram (function) juga berhasil memastikan implementasi kode pencarian ini terenkapsulasi secara rapi, mencegah redundansi, dan lebih mudah digunakan berulang kali di berbagai alur pemrosesan data.